RepoChat Test Plan

Objective:
Verify that RepoChat's core flows work correctly and fail gracefully Ingesting a GitHub repository, answering questions about it, rejecting irrelevant questions, caching answers, and managing storage. This plan documents what the automated suite already covers and where manual and exploratory testing should fill the gaps.

Scope:

In scope
• Repository ingestion: URL validation, GitHub access, cloning, file parsing, chunking, embeddings, progress tracking
• Query handling: relevance guard, query interpretation, retrieval, reranking, post-retrieval filtering, context compression, answer generation
• Semantic cache: hits, misses, similarity threshold, invalidation on re-ingest
• Storage management: limits, critical threshold, oldest-repo cleanup, database compaction
• End-to-end user flows in the browser: ingest a repo, ask questions, view chat history

Out of scope
• Login and sign-up (not part of RepoChat)
• Load, stress, and penetration testing
• The internals of third-party services (GitHub, OpenAI, MongoDB)

Test levels:

Level	Tool	Tests	Purpose
Unit	Jest (TypeScript)	104	Verify single services, controllers, and handlers in isolation
Integration	Jest with an in-memory MongoDB	21	Verify services working together with a real database layer
End-to-End	Playwright (Chromium)	6	Verify real user flows through the frontend
Manual / Exploratory	Browser, Jira	planned	Cover gaps the automation does not (see Coverage gaps and manual test candidates)

Test areas:

Area	Cases	Techniques used
Query handling (controller, handler, guards, interpreter, builder, processor, query pipeline)	27	Equivalence partitioning (relevant vs irrelevant queries), decision tables (cache hit or miss, embedding present, filter result), negative testing (provider errors)
Chunk processing and vector search	9	Negative testing (missing embedding, repository and provider failures), error handling and error wrapping
Retrieval ranking and filtering (reranker, post-retrieval filter, context compression)	15	Boundary value analysis (topK, score threshold, compression threshold, per-file

		cap), equivalence partitioning
Semantic cache	9	Boundary value analysis (similarity threshold), state transition (miss, hit, invalidate), decision tables
Ingestion flow (ingest service, controller, handler, ingest pipeline)	18	State transition (ingestion stages, terminal states), equivalence partitioning (up-to-date vs new SHA), negative testing
Repository and GitHub access (GitHub service, repo processing, files)	22	Equivalence partitioning (reachable, private, missing, unreachable repo), negative testing, error guessing
Ingest progress tracking	10	State transition (stage order, replace, terminal error), equivalence partitioning
Storage management	11	Boundary value analysis (storage limit, critical threshold, safe buffer), decision tables
Smoke and infrastructure	5	Smoke testing
End-to-end user flows	5	State transition (page flow), equivalence partitioning (valid vs invalid repo, relevant vs irrelevant question), negative testing
Total	131	

The detailed cases are in the companion document, RepoChat Test Cases

Test data and environment:

• Automated tests: Unit tests mock all providers. Integration tests use an in-memory MongoDB and mock OpenAI. End-to-end tests run against [mocked / real] backend responses. Unit tests mock all providers. Integration tests use an in-memory MongoDB and mock OpenAI. End-to-end tests run against [mocked / real] backend responses
• Browser for end-to-end tests: Chromium, one worker
• Additional browsers or devices for manual testing: 375px mobile viewport using Chrome DevTools
• Test repositories (small): small - a repo I own
• Deployed URL or local build under test: Local build at http://localhost:3000 and/or production URL

Entry and exit criteria:

Entry
• Backend and frontend run without startup errors
• Test data and repositories are prepared
• The automated suite runs and the smoke tests pass
Exit
• All automated tests pass (or each failure has a logged bug)
• All high-priority cases are executed
• No open critical or high-severity bugs without a documented decision
• Manual and exploratory findings are logged in Jira

Current automated results:

Suite	Result	Run time
Unit and integration (26 suites)	125 of 125 passed	19.4 seconds

End-to-end (6 tests)	6 of 6 passed	28.6 seconds

Coverage gaps and manual test candidates:

The automated suite is strong on backend logic. These areas have little or no automated coverage and are the best targets for manual and exploratory testing

• Chat input behavior: empty messages, very long messages, special characters, emoji, pasted code, markup or script text, rapid double-send
• State edge cases in the UI: refresh mid-reply, switching repositories mid-reply, using the back button during ingestion
• Error handling in the real UI: private repository, nonexistent repository, network failure while chatting, whether typed text is kept after a failure
• Loading and progress states: what the user sees at each ingestion stage and on slow responses
• Repository variety: large repositories, different languages, repositories with unsupported files only
• Cross-browser and responsive layout: only Chromium is covered, and no mobile viewport is tested
• Concurrency: ingesting the same repository twice at once, or a query during re-ingestion
• Security and safety: prompt-injection style questions, handling of markup in messages
• Accessibility: keyboard navigation and screen reader basics
• Real-service behavior: whether answers behave sensibly against the real providers, since much of the suite relies on mocks